

UNIVERSIDADE DO ESTADO DO PARÁ - UEPA
CURSO ENGENHARIA DE SOFTWARE
DISCIPLINA ARQUITETURA DE SOFTWARE
PROFª JACQUELINE TEIXEIRA

Atividade Prática: Refatoração Arquitetural e Princípios SÓLIDO

Curso: Bacharelado Engenharia de Software

Disciplina: Arquitetura de Software

Linguagem Base do Modelo: TypeScript (Node.js)

1. Contextualização e Análise do Problema Legado

No desenvolvimento de software, a rigidez do código surge frequentemente devido ao alto acoplamento e à baixa coesão. Quando uma classe assume responsabilidades de infraestrutura (como abrir conexões de banco de dados ou enviar e-mails via protocolo SMTP) e instanciar diretamente essas dependências através do operador `new`, surgem severas limitações arquiteturais.

Análise Técnica de Violações:

Princípio / Conceito	Violação Identificada no Código Legado	Impacto Negativo na Aplicação
DIP (Inversion Principle)	GeradorRelatorio depende diretamente de classes concretas (MySQLConnection, SmtEmailSender).	Impossibilita a troca do banco ou do meio de notificação sem alterar o código do gerador.
OCP (Open/Closed Principle)	Para adicionar um novo meio de envio (ex: Slack), é necessário modificar os métodos da classe principal.	Aumenta o risco de regressão em funcionalidades estáveis a cada mudança de infraestrutura.
SRP (Single Responsibility)	A classe gera relatórios, busca dados no banco e orquestra o protocolo de e-mail.	Baixa coesão; a classe muda por motivos de regra de negócio ou por mudanças na infraestrutura.
Testabilidade	Uso direto de <code>new</code> internamente impede a substituição das dependências por Mocks/Stubs.	Impossibilita testes unitários isolados em ambiente de Integração Contínua.

2. Código Legado (NÃO REFATORADO)

Abaixo está o código TypeScript que representa a situação inicial do sistema antes da refatoração.

```
//  
=====
```

```
// CÓDIGO LEGADO - NAO REFATORADO
```

```
// =====
```

```
class MySqlConnection {  
  public buscarDadosFinanceiros(): string[] {  
    console.log("[INFRA] Conectando ao MySQL e buscando lançamentos...");  
    return ["Receita: R$ 15.000,00", "Despesa: R$ 8.200,00", "Lucro: R$ 6.800,00"];  
  }  
}
```

```
class SmtperEmailSender {  
  public enviarEmail(destino: string, mensagem: string): void {  
    console.log(`[INFRA] Enviando e-mail via SMTP para ${destino}: ${mensagem}`);  
  }  
}
```

```
export class GeradorRelatorioLegado {  
  public processarEMandar(destino: string): void {  
    // VIOLACAO: Acoplamento rígido via operador 'new'  
    const banco = new MySqlConnection();  
    const emailSender = new SmtperEmailSender();  
  
    // VIOLACAO: Baixa coesão (orquestra regra de negócio e infraestrutura)  
    const dados = banco.buscarDadosFinanceiros();  
    const relatorioFormatado = `--- RELATÓRIO FINANCEIRO ---\n${dados.join("\n")}`;  
  
    emailSender.enviarEmail(destino, relatorioFormatado);  
  }  
}
```

3. Template do Arquivo README.md

A estrutura abaixo orienta como elaborar a documentação e a justificativa técnica exigida na entrega.

Atividade Prática: Refatoração de Arquitetura de Software

Aluno(a): [Nome do Aluno]

Matrícula: [Número de Matrícula]

Disciplina: Arquitetura de Software - Prof. Jacqueline Teixeira

1. Análise do Código Legado e Violações Encontradas

No código original fornecido, observamos os seguintes problemas estruturais:

Alto Acoplamento Concreto:

Violação do DIP:

Violação do OCP:

Impossibilidade de Testes Unitários:

2. Mudanças Efetuadas e Justificativa Técnica

1. Criação de Contratos (Interfaces):

Justificativa:

2. Injeção de Dependência (DI):

Justificativa:

3. Extensão via Princípio Aberto/Fechado (OCP):

Justificativa:

3. Como Executar o Projeto

Instalar dependências

npm install

Compilar e executar o código refatorado

npx ts-node src/index.ts